Práctica no.1

Conociendo tus datos

Sofía Cristina Durán Muñoz [22211752]
Departamento de Ingeniería Eléctrica y Electrónica
Tecnológico Nacional de México / Instituto Tecnológico de Tijuana

Febrero 26, 2026

Parte I. Carga y Limpieza de Datos

Paso 1

Para el desarrollo del análisis estadístico se trabajó en un entorno de Jupyter Notebook utilizando Visual Studio Code. La Figura 1 muestra que se importaron las librerías Numpy, Pandas, Matplotlib y Seaborn, las cuales permiten la manipulación, limpieza y análisis de datos, así como la visualización gráfica de los resultados. Estas herramientas fueron empleadas para analizar la base de datos *Parkinsons Telemonitoring* disponible en el repositorio **UCI Machine Learning**, facilitando el procesamiento y la exploración de la información.

Carga y limpieza de datos

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Figura 1.- Importación de librerías.

Paso 2

Posteriormente, se procedió a la carga del conjunto de datos directamente desde el repositorio oficial de UCI mediante su URL, utilizando la función *read_csv()* de la librería **Pandas**. Una vez cargados los datos en un *DataFrame*, se mostraron las primeras filas para verificar la correcta importación, como se ilustra en la Figura 2.

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/parkinsons/telemonitoring/parkinsons_updrs.data"
df = pd.read_csv(url)
display(df.head())
```

Figura 2.- Cargar el dataset.

Paso 3

Después, se llevó a cabo el proceso de limpieza de datos. La Figura 3 muestra que se identificaron posibles valores faltantes reemplazando los caracteres “?” por valores nulos.

Posteriormente, se contabilizaron los valores faltantes por columna para evaluar la calidad del conjunto de datos. Finalmente, se eliminaron los registros incompletos con el método *dropna()*, y se compararon las dimensiones del DataFrame antes y después de la limpieza para verificar el impacto del proceso de depuración.

```
dimensiones_antes = df.shape
df = df.replace('?', np.nan)
valores_faltantes = df.isna().sum()
print("Valores faltantes por columna:")
print(valores_faltantes)
print("\n")
df = df.dropna()
dimensiones_despues = df.shape
print("Resumen de Dimensiones:")
print(f"Dimensiones ANTES de la limpieza: {dimensiones_antes}")
print(f"Dimensiones DESPUÉS de la limpieza: {dimensiones_despues}")
```

Figura 3.- Limpieza de los datos.

Resultados de Parte I

En la Figura 4a se realizó la carga y revisión inicial del conjunto de datos. Al visualizar las primeras cinco filas, se confirmó que la base contiene 22 columnas con variables relacionadas con la enfermedad de Parkinson.

Posteriormente, se evaluó la presencia de valores faltantes en cada columna, observándose que todas presentan un total de cero valores nulos. En la Figura 4b se muestra que al comparar las dimensiones del DataFrame antes y después del proceso de limpieza, se comprobó que no hubo cambios en el número de registros ni en el número de variables, lo que indica que el conjunto de datos se encontraba completo y no requirió eliminación.

	subject#	age	sex	test_time	motor_UPDRS	total_UPDRS	Jitter(%)	Jitter(Abs)	Jitter:RAP	Jitter:PPQ5	...
0	1	72	0	5.6431	28.199	34.398	0.00662	0.000034	0.00401	0.00317	...
1	1	72	0	12.6660	28.447	34.894	0.00300	0.000017	0.00132	0.00150	...
2	1	72	0	19.6810	28.695	35.389	0.00481	0.000025	0.00205	0.00208	...
3	1	72	0	25.6470	28.905	35.810	0.00528	0.000027	0.00191	0.00264	...
4	1	72	0	33.6420	29.187	36.375	0.00335	0.000020	0.00093	0.00130	...

5 rows × 22 columns

a)

```
Valores faltantes por columna
subject#        0
age             0
sex             0
test_time       0
motor_UPDRS     0
total_UPDRS     0
Jitter(%)       0
Jitter(Abs)     0
Jitter:RAP      0
Jitter:PPQ5     0
Jitter:DDP      0
```

b)

Figura 4.- Resultados de Parte I.

Parte II. Identificación de Atributos

Paso 4

Se analizaron los tipos de datos detectados automáticamente por la librería **Pandas** mediante el atributo *dtypes*, como se muestra en la Figura 5.

```
#Tipos detectados por pandas
tipos_pandas = df.dtypes
print("Tipos de datos detectados por Pandas:")
print(tipos_pandas)
```

Figura 5.- Revisión de tipos detectados.

De acuerdo con la Tabla I, se concluye que las variables subject#, age y sex fueron identificadas como **datos enteros** (int64), mientras que el resto de las variables fueron reconocidas como **datos numéricos de tipo decimal** (float64).

Tabla I.- Relación de tipos de datos.

Tipos de datos detectados			
Dato	**Tipo**	**Dato**	**Tipo**
subject#	int64	Shimmer:APQ3	float64
age	int64	Shimmer:APQ5	float64
sex	int64	Shimmer:APQ11	float64
test_time	float64	Shimmer:DDA	float64
motor_UPDRS	float64	NHR	float64
total_UPDRS	float64	HNR	float64
Jitter(%)	float64	RPDE	float64
Jitter(Abs)	float64	DFA	float64
Jitter:RAP	float64	PPE	float64
Jitter:PPQ5	float64	Shimmer	float64
Jitter:DDP	float64	Shimmer (dB)	float64

Esto indica que el conjunto de datos está compuesto principalmente por variables cuantitativas continuas, especialmente aquellas relacionadas con mediciones acústicas y evaluaciones clínicas (*motor_UPDRS*, *total_UPDRS*), lo cual es adecuado para realizar análisis estadísticos. Además, no se detectaron variables de tipo categórico o texto, lo que facilita el procesamiento matemático y estadístico de la información.

Paso 5

Justificación de cinco clasificaciones:

1. **sex (*Binario*)**: Es un atributo con solo dos estados. En este dataset, biológicamente se registra con dos estados (0 y 1), por lo tanto, se define como binario, aunque *Pandas* lo lea como un número entero (int64).
2. **age (*Numérico*)**: Es una cantidad cuantitativa que se representa con números enteros. Aunque *Pandas* lo lee correctamente como numérico (int64), teóricamente es un numérico de tipo discreto o continuo.

3. **subject# (*Nominal*)**: Son valores que representan categorías (el número de paciente) sin un orden lógico matemático. Es decir, solo es un identificador. *Pandas* lo detecta como int64 porque usa números, pero teóricamente funciona como un nombre.
4. **total_UPDRS (*Numérico*)**: Es la puntuación de la escala de Parkinson. Es una cantidad cuantitativa con valores reales (tiene decimales), y *Pandas* lo identifica correctamente como float64.
5. **Jitter(%) (*Numérico*)**: Representa una medición física de la frecuencia de la voz. Al ser una cantidad cuantitativa que acepta infinitos valores posibles en su rango (continuo), encaja perfectamente con su tipo en *Pandas* (float64).

Resultados de la Parte II

En esta etapa se realizó una comparación entre los tipos de datos detectados automáticamente por *Pandas* y la clasificación teórica de cada variable según su naturaleza estadística. Para ello, se definió manualmente el tipo teórico y se construyó una tabla que integra el nombre de la variable, su tipo teórico, el tipo detectado por Pandas y el número de valores únicos presentes en cada columna.

La Figura 6 muestra que la variable *subject#* es de tipo nominal, ya que identifica a cada paciente, aunque Pandas la reconoce como entero. La variable *sex* fue clasificada teóricamente como binaria, aunque también es detectada como entero. Por otro lado, el resto de las variables corresponden a mediciones numéricas continuas.

	Nombre del atributo	Tipo teórico	Tipo detectado por pandas	Número de valores únicos
0	subject#	Nominal	int64	42
1	age	Numérico	int64	23
2	sex	Binario	int64	2
3	test_time	Numérico	float64	2442
4	motor_UPDRS	Numérico	float64	1080
5	total_UPDRS	Numérico	float64	1129
6	Jitter(%)	Numérico	float64	1305
7	Jitter(Abs)	Numérico	float64	4105
8	Jitter:RAP	Numérico	float64	853
9	Jitter:PPQ5	Numérico	float64	840
10	Jitter:DDP	Numérico	float64	1703
11	Shimmer	Numérico	float64	3581
12	Shimmer(dB)	Numérico	float64	852
13	Shimmer:APQ3	Numérico	float64	2664
14	Shimmer:APQ5	Numérico	float64	2850
15	Shimmer:APQ11	Numérico	float64	3283
16	Shimmer:DDA	Numérico	float64	4223
17	NHR	Numérico	float64	5532
18	HNR	Numérico	float64	4780
19	RPDE	Numérico	float64	5430
20	DFA	Numérico	float64	5282
21	PPE	Numérico	float64	4777

Figura 6.- Clasificación teórica.

Parte III. Estadística Descriptiva

Paso 6

Se utilizó el método *describe()* de la librería **Pandas**. Esta función permitió obtener medidas de tendencia central y dispersión para cada variable numérica, incluyendo el número de observaciones (*count*), media (*mean*), desviación estándar (*std*), valores mínimos y máximos, así como los cuartiles (25%, 50% y 75%).

Los resultados que se muestran en la Figura 7 ilustran que el conjunto de datos está compuesto por 5,875 registros. La edad promedio de los pacientes es aproximadamente 64.8 años, con un rango entre 36 y 85 años. En cuanto a las variables clínicas, el promedio de *motor_UPDRS* es cercano a 21.3 y el de *total_UPDRS* a 29.0, lo que refleja distintos niveles de severidad en los pacientes evaluados.

```
estadisticas = df.describe()
display(estadisticas)
```

	subject#	age	sex	test_time	motor_UPDRS	total_UPDRS	Jitter(%)	Jitter(Abs)	Jitter:RAP	Jitter:PPQ5	...
count	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	...
mean	21.494128	64.804936	0.317787	92.863722	21.296229	29.018942	0.006154	0.000044	0.002987	0.003277	...
std	12.372279	8.821524	0.465656	53.445602	8.129282	10.700283	0.005624	0.000036	0.003124	0.003732	...
min	1.000000	36.000000	0.000000	-4.262500	5.037700	7.000000	0.000830	0.000002	0.000330	0.000430	...
25%	10.000000	58.000000	0.000000	46.847500	15.000000	21.371000	0.003580	0.000022	0.001580	0.001820	...
50%	22.000000	65.000000	0.000000	91.523000	20.871000	27.576000	0.004900	0.000035	0.002250	0.002490	...
75%	33.000000	72.000000	1.000000	138.445000	27.596500	36.399000	0.006800	0.000053	0.003290	0.003460	...
max	42.000000	85.000000	1.000000	215.490000	39.511000	54.992000	0.099990	0.000446	0.057540	0.069560	...

8 rows × 22 columns

Figura 7.- Estadística general.

Este análisis descriptivo proporciona una visión general del comportamiento estadístico de las variables y constituye la base para análisis posteriores más específicos.

Paso 7

En la Figura 8 se calcularon medidas de tendencia central para las variables *age* y *total_UPDRS*. En el caso de la edad, los resultados muestran que la media y la mediana son valores muy cercanos, lo que sugiere una distribución simétrica en la muestra de pacientes.

```
 Tendencia Central para la variable 'age'
Media: 64.80
Mediana: 65.0
Moda: 58

 Tendencia Central para la variable 'total_UPDRS'
Media: 29.02
Mediana: 27.576
Moda: 32.0
```

Figura 8.- Tendencia central.

Para la variable *total_UPDRS*, que mide la severidad total de los síntomas de la enfermedad de Parkinson, también se calcularon media, mediana y moda. La cercanía entre la media y la mediana sugiere que no existe una asimetría extrema, indicando diferencias en los niveles de progresión de la enfermedad entre los pacientes.

Paso 8

Posteriormente, se analizaron medidas de dispersión para *total_UPDRS*, incluyendo el rango, los cuartiles (Q1 y Q3), el rango intercuartílico (IQR), la varianza y la desviación estándar, como se ilustra en la Figura 9.

```
 Medidas de Dispersión para la variable 'total_UPDRS'
Rango: 47.99
Q1 (Primer Cuartil, 25%): 21.37
Q3 (Tercer Cuartil, 75%): 36.40
IQR (Rango Intercuartílico): 15.03
Varianza: 114.50
Desviación estándar: 10.70
```

Figura 9.- Medidas de dispersión.

El rango permitió identificar la amplitud total de los valores observados, mientras que el IQR mostró la dispersión del 50% central de los datos. La varianza y la desviación estándar confirmaron la presencia de variabilidad en la severidad de los síntomas.

Paso 9

Finalmente, la Figura 10 ilustra el ***análisis de frecuencias*** para variables categóricas.

- En el caso de *subject#*, se identificaron los diez pacientes con mayor número de registros.
- Se analizó la variable *sex*, observando la distribución de pacientes según su clasificación binaria, lo que permite conocer la proporción de cada grupo dentro de la muestra estudiada.

```
 Top-10 de pacientes con más registros en el dataset

subject#
29    168
35    165
41    165
34    161
7     161
24    156
5     156
6     156
9     152
8     150
Name: count, dtype: int64


 Frecuencia de la variable categórica 'sex'

sex
0    4008
1    1867
Name: count, dtype: int64
```

Figura 10.- Análisis de frecuencias.

Parte IV. Visualización de Datos

Paso 10 y 11

Se realizó una visualización gráfica de la variable *age* para examinar su distribución. Para ello, se utilizó la librería **Seaborn** con un estilo visual limpio (*whitegrid*), generando dos gráficos complementarios: un histograma y una gráfica de densidad (KDE).

La Figura 11 ilustra el histograma: se observa la frecuencia de registros en cada intervalo de edad, mostrando cómo se distribuyen los pacientes dentro de distintos rangos etarios.

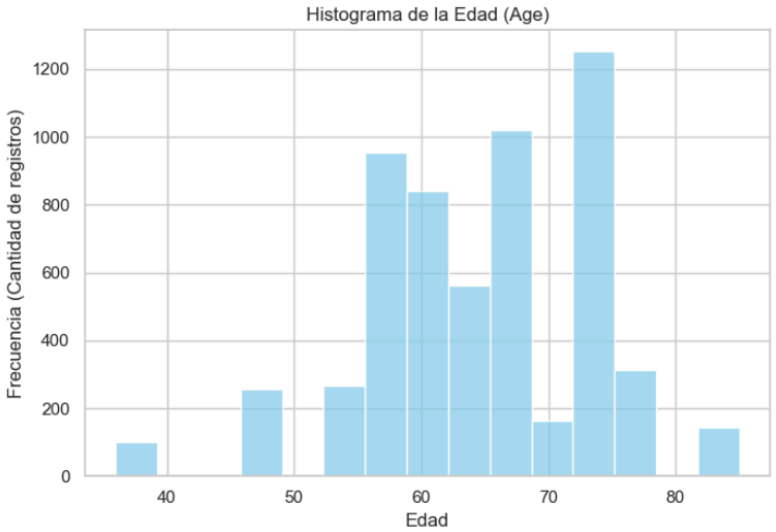

Figura 11.- Histograma de la variable age.

Por su parte, la gráfica de densidad mostrada en la Figura 12 proporcionó una representación suavizada de la distribución, facilitando la identificación de posibles concentraciones o patrones en los datos.

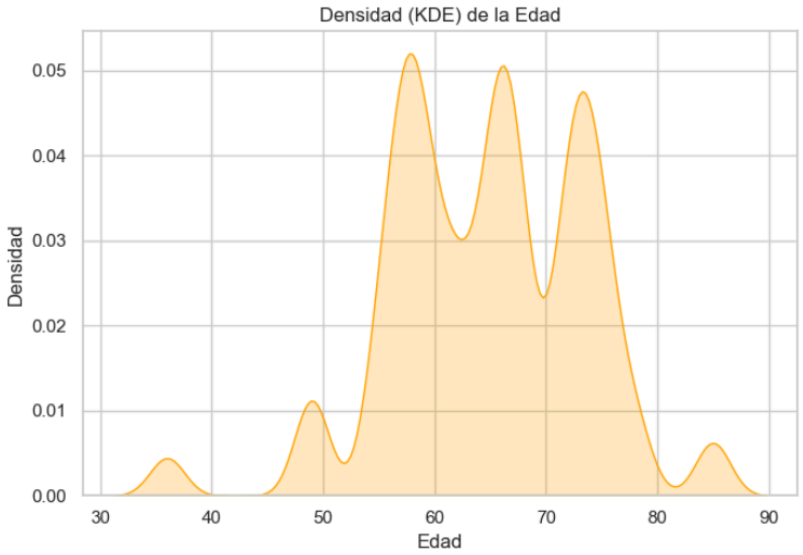

Figura 12.- Densidad de la variable age.

En conjunto, ambas visualizaciones permiten analizar la forma de la distribución (simetría, concentración y dispersión), así como identificar si la variable presenta sesgo o agrupaciones predominantes en ciertos rangos de edad.

Paso 12

Se elaboraron boxplots para las variables *age*, *motor_UPDRS* y *total_UPDRS*, ilustrados en la Figura 13. Esta representación permitió observar la mediana, los cuartiles y la amplitud intercuartílica de cada variable, así como detectar valores atípicos.

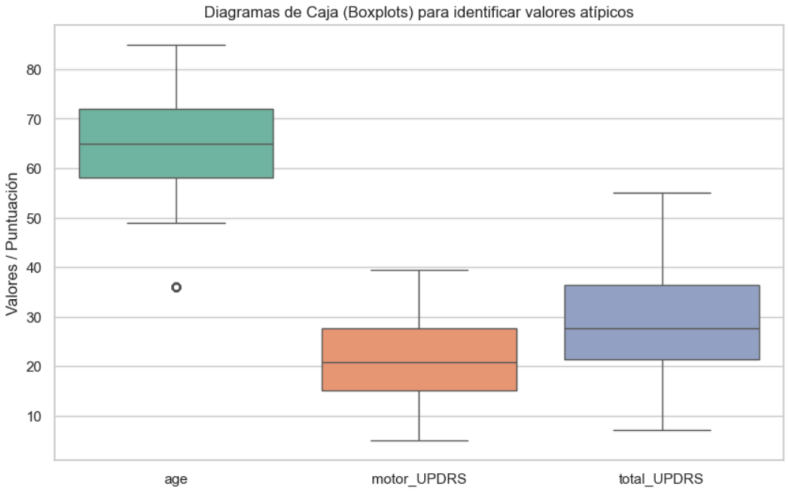

Figura 13.- Boxplot del dataset.

Paso 13

Posteriormente, se construyó un gráfico de dispersión entre *age* y *total_UPDRS*, con el objetivo de explorar la relación entre la edad de los pacientes y la severidad total de los síntomas de la enfermedad. La Figura 14 permitió identificar la tendencia general de los datos y evaluar visualmente si existe algún patrón.

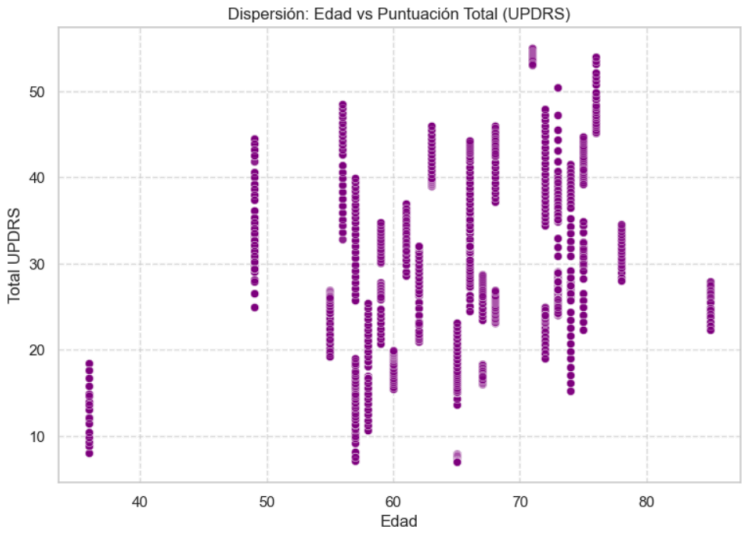

Figura 14.- Dispersión de la variable *Total_UPDRS*.

Paso 14

Se elaboró una gráfica de barras para representar a los diez pacientes con mayor número de registros dentro del conjunto de datos. Para ello, se utilizó la función *value_counts()* sobre la variable *subject#*. La Figura 15 muestra claramente la frecuencia de registros por paciente, evidenciando que el dataset contiene múltiples evaluaciones realizadas a los mismos sujetos en distintos momentos de seguimiento.

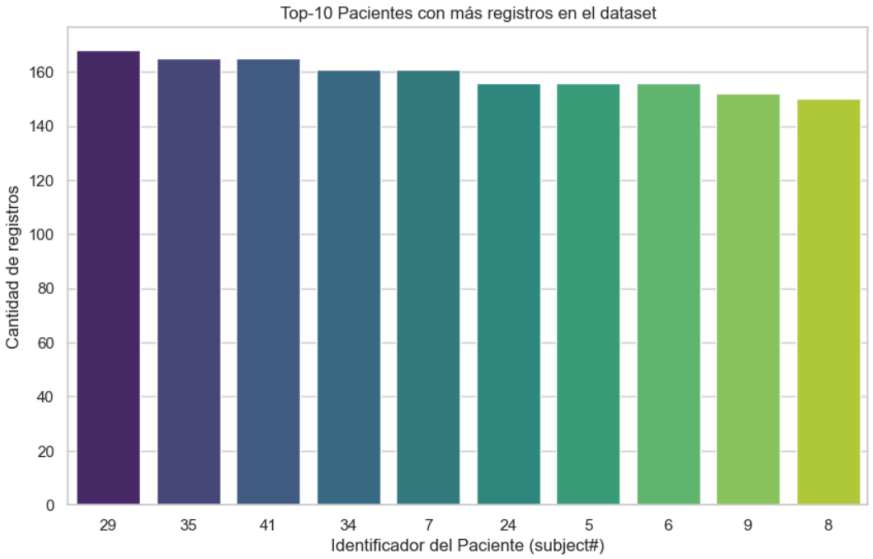

Figura 15.- Gráfica de barras.

Este tipo de visualización facilita la identificación de pacientes con mayor cantidad de observaciones, lo cual puede ser relevante en análisis posteriores, especialmente en estudios de progresión de la enfermedad o modelado predictivo.

Parte V. Medición de Similitud y Disimilitud

Paso 15

La Figura 16 ilustra que se realizó la selección de una muestra aleatoria de 10 registros del conjunto de datos utilizando el método *sample()* de la librería **Pandas**. Se estableció el parámetro *random_state* con el fin de que la misma muestra pueda obtenerse nuevamente si se ejecuta el código bajo las mismas condiciones. La visualización de esta muestra permitió comprobar la estructura de los datos seleccionados.

```
print(" Paso 15: Selección de muestra (10 filas) ")
# Seleccionamos 10 filas
muestra = df.sample(10, random_state=42)
# Mostramos la muestra para comprobar
display(muestra)
```

Figura 16.- Selección de 10 muestras aleatorias.

5.1 Distancias Numéricas

Paso 16

Se implementó la función de distancia de Minkowski con el objetivo de medir la similitud entre dos observaciones de la muestra seleccionada. Para ello, se consideraron las variables numéricas *age*, *motor_UPDRS* y *total_UPDRS*, extrayendo sus valores en forma de vectores. Se definió una función que calcula la distancia de **Minkowski** entre dos vectores, permitiendo modificar el parámetro. La Figura 17 muestra que se calculó la distancia que corresponde a la métrica **Manhattan**, la cual suma las diferencias absolutas entre los valores de cada variable y la distancia **Euclidiana**, que representa la distancia geométrica tradicional en el espacio tridimensional definido por las variables seleccionadas. Esto permitió cuantificar numéricamente qué tan similares o diferentes son estos pacientes con respecto a su edad y puntuaciones clínicas.

```
 Paso 16: Función Minkowski
Distancia Manhattan (p=1) entre paciente 0 y 1: 87.43
Distancia Euclidiana (p=2) entre paciente 0 y 1: 50.64
```

Figura 17.- Cálculo de distancias numéricas.

Paso 17

Se construyó una matriz de distancias de tamaño 10×10 utilizando la distancia **Euclidiana**. Primero, se creó una matriz cuadrada inicializada en ceros y posteriormente se calcularon las distancias. La Figura 18 muestra la matriz resultante, donde la diagonal principal contiene ceros. Finalmente, la matriz fue visualizada mediante un mapa de calor utilizando *imshow()*, lo que permitió identificar los niveles de similitud entre los pacientes.

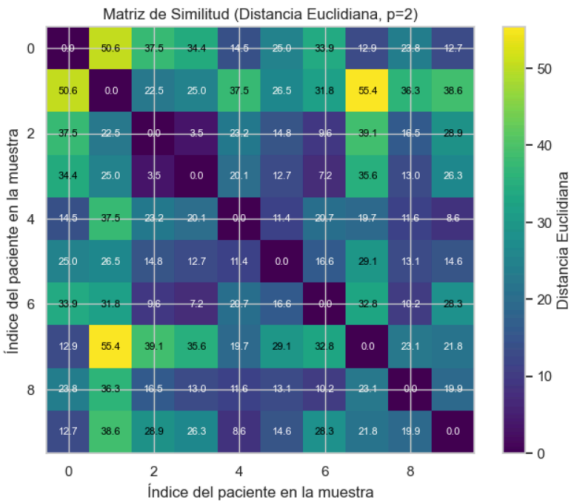

Figura 18.- Matriz de similitud.

5.2 Distancias para Binarios

Se analizaron medidas de distancia y similitud para variables binarias.

Paso 18

Para ello, se creó una nueva variable denominada *alto_UPDRS*, la cual toma el valor 1 cuando la puntuación *total_UPDRS* es mayor a 30 y 0 en caso contrario. De esta manera, se transformó una variable numérica en una variable categórica binaria que indica un nivel alto de severidad.

Posteriormente, se seleccionaron las variables binarias *sex* y *alto_UPDRS* para comparar dos observaciones de la muestra. Se implementó primero la distancia de Hamming para cuantificar qué tan distintos son dos pacientes considerando únicamente características categóricas binarias, como se muestra en la Figura 19.

```
# Creamos una copia de la muestra para no alterar la original
muestra_bin = muestra.copy()
# Creamos la variable 'alto_UPDRS': 1 si es > 30, 0 si es <= 30
muestra_bin['alto_UPDRS'] = (muestra_bin['total_UPDRS'] > 30).astype(int)
columnas_binarias = ['sex', 'alto_UPDRS']
datos_binarios = muestra_bin[columnas_binarias].values
fila_0_bin = datos_binarios[0]
fila_1_bin = datos_binarios[1]

print(f"Fila 0 (Atributos Binarios): {fila_0_bin}")
print(f"Fila 1 (Atributos Binarios): {fila_1_bin}\n")


# Paso 18: Implementar Hamming
print("Paso 18: Distancia de Hamming")
def distancia_hamming(x, y):
    """Calcula la proporción de diferencias entre dos vectores binarios"""
    diferencias = np.sum(x != y)
    return diferencias / len(x)

dist_hamming = distancia_hamming(fila_0_bin, fila_1_bin)
print(f"Distancia de Hamming entre Fila 0 y Fila 1: {dist_hamming:.2f}")
```

Figura 19.- Implementación de Hamming.

Paso 19

Después, se implementó la similitud de Jaccard, que evalúa la proporción de coincidencias en los valores iguales a 1 respecto al total de casos donde al menos uno de los dos registros presenta un 1. Esta medida es especialmente útil cuando se desea analizar la coincidencia en la presencia de una característica específica. La Figura 20 ilustra claramente que no se encontró similitud.

```
Paso 19: Similitud de Jaccard
Similitud de Jaccard entre Fila 0 y Fila 1: 0.00
```

Figura 20.- Resultados de la similitud de Jaccard.

5.3 Similitud Coseno

Paso 20

En esta etapa se calculó la similitud del coseno entre las variables numéricas *age*, *motor_UPDRS* y *total_UPDRS*.

Primero, las variables fueron estandarizadas mediante el cálculo del z-score, lo que permitió transformar los datos a una escala con media 0 y desviación estándar 1.

Posteriormente, se implementó la función de similitud del coseno, la cual mide el ángulo entre dos vectores en el espacio multidimensional. A diferencia de la distancia Euclidiana, esta métrica no evalúa la magnitud absoluta de las diferencias, sino la orientación relativa de los vectores. Valores cercanos a 1 indican alta similitud, valores cercanos a 0 indican baja relación, y valores negativos reflejan direcciones opuestas.

Finalmente, se construyó una matriz de similitud de tamaño 10×10 y se visualizó mediante un mapa de calor, lo que permitió identificar visualmente qué pacientes presentan patrones clínicos más semejantes considerando simultáneamente edad y puntuaciones UPDRS.

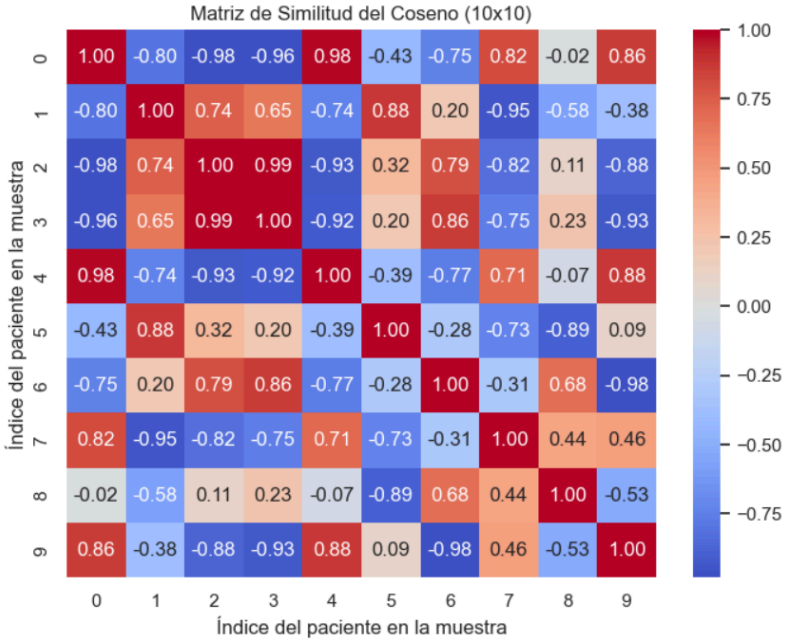

Figura 21.- Matriz de similitud del coseno.

5.4 Distancia para Tipos Mixtos

Se calculó una matriz de distancia para datos de tipos mixtos, combinando variables numéricas y categóricas dentro de una misma medida de similitud.

En primer lugar, las variables numéricas fueron normalizadas mediante el método **Min-Max**, transformando sus valores a un rango entre 0 y 1. Este procedimiento fue necesario para evitar que variables con escalas mayores influyeran desproporcionadamente. Posteriormente, se definió una función de distancia mixta que combina dos componentes:

- Para las variables numéricas, se calculó la diferencia entre los valores normalizados.
- Para las variables categóricas, se asignó un valor de 0 cuando los atributos coinciden y 1 cuando son diferentes.

La distancia total se obtuvo promediando ambas contribuciones, considerando el total de variables analizadas. Finalmente, se construyó una matriz de distancia de tamaño 10×10 y se visualizó mediante un mapa de calor, lo que permitió identificar visualmente los niveles de similitud entre los pacientes. La Figura 22 muestra que valores cercanos a 0 indican mayor semejanza, mientras que valores más próximos a 1 representan mayor diferencia considerando simultáneamente características clínicas, demográficas y categóricas.

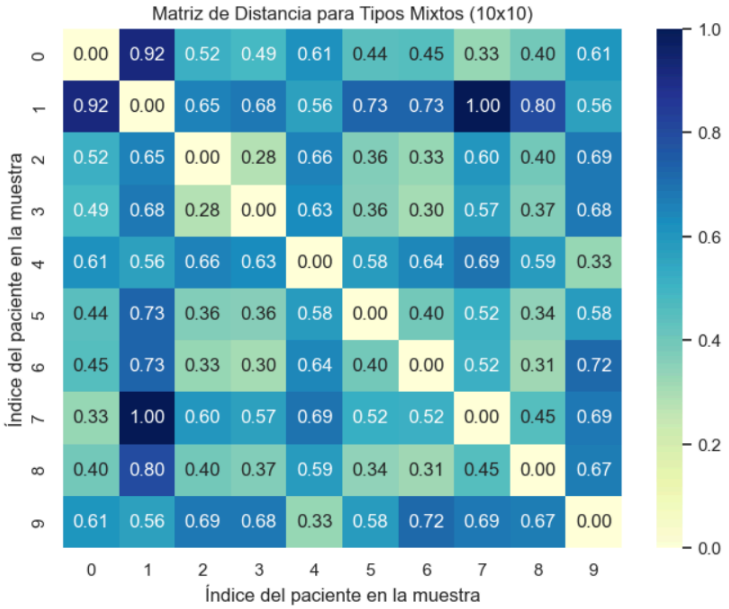

Figura 22.- Distancia para tipos mixtos.